

ICPC Notebook Java

Team Reference

Convenciones: Java 17; indices desde cero; intervalos semiabiertos salvo indicacion contraria. Los fragmentos se compilan y prueban desde el repositorio.

Índice

1. Plantilla y entrada **1**
1.1. Main de concurso . . . 1
1.2. Entrada rapida . . . 2

2. Estructuras de datos **3**
2.1. DSU, Fenwick y segment tree . . . 3

3. Busqueda y ordenamiento **4**
3.1. Busqueda binaria, ternaria y counting sort . . . 4

4. Programacion dinamica **5**
4.1. Patrones de programacion dinamica . . . 5

5. Cadenas **6**
5.1. KMP, Manacher, Boyer-Moore y Z . . . 6
5.2. Trie y rolling hash . . . 8

6. Matematicas **9**
6.1. Teoria de numeros . . . 9
6.2. Combinatoria . . . 11

7. Geometria **11**
7.1. Geometria plana . . . 11

8. Grafos **13**
8.1. Recorridos y caminos . . . 13
8.2. SCC, Bellman-Ford, LCA y flujo maximo . . . 14
8.3. Puentes y puntos de articulacion . . . 16

9. Teoria de juegos **17**
9.1. Nim, Grundy y minimax . . . 17

1. Plantilla y entrada

1.1. Main de concurso

Archivo unico listo para un juez en linea.

```java
import java.io.*;
import java.util.*;

/** Standalone template for judges that require a single Main.java file. */
```

```java
public class Main {
    static final class FastScanner {
        private final InputStream in = new BufferedInputStream(System.in);
        private final byte[] buffer = new byte[1 << 16];
        private int pos, len;

        private int read() throws IOException {
            if (pos >= len) {
                len = in.read(buffer);
                pos = 0;
                if (len < 0) return -1;
            }
            return buffer[pos++] & 0xff;
        }

        String next() throws IOException {
            int c;
            do { c = read(); } while (c >= 0 && c <= ' ');
            if (c < 0) return null;
            StringBuilder token = new StringBuilder();
            while (c > ' ') { token.append((char) c); c = read(); }
            return token.toString();
        }

        int nextInt() throws IOException { return Integer.parseInt(next()); }
        long nextLong() throws IOException { return Long.parseLong(next()); }
    }

    public static void main(String[] args) throws Exception {
        FastScanner in = new FastScanner();
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        // Solve the problem here. Read with in.nextInt(), write with out.println().
        out.flush();
    }
}
```

1.2. Entrada rapida

Tokens ASCII; devuelve null al terminar la entrada.

```java
package notebook;

import java.io.BufferedInputStream;
import java.io.IOException;
import java.io.InputStream;

/** Fast byte input for ASCII contest tokens. Use PrintWriter/StringBuilder for output. */
public final class ContestIO {
    private final InputStream in;
    private final byte[] buffer = new byte[1 << 16];
    private int pos, len;

    public ContestIO(InputStream input) { in = new BufferedInputStream(input); }

    private int read() throws IOException {
        if (pos >= len) {
            len = in.read(buffer);
            pos = 0;
            if (len < 0) return -1;
        }
        return buffer[pos++] & 0xff;
    }

    /** Returns null at EOF. Tokens are decoded as ASCII. */
    public String next() throws IOException {
        int c;
        do { c = read(); } while (c >= 0 && c <= ' ');
        if (c < 0) return null;
        StringBuilder token = new StringBuilder();
        while (c > ' ') { token.append((char) c); c = read(); }
        return token.toString();
    }

    public int nextInt() throws IOException { return Integer.parseInt(next()); }
    public long nextLong() throws IOException { return Long.parseLong(next()); }
}
```

2. Estructuras de datos

2.1. DSU, Fenwick y segment tree

Indices desde cero e intervalos semiabiertos.

```java
package notebook;

/** Zero-indexed contest data structures. */
public final class DataStructures {
    private DataStructures() {}

    /** Disjoint set union with path compression and union by size. */
    public static final class DSU {
        private final int[] parent, size;
        private int components;
        public DSU(int n) {
            if (n < 0) throw new IllegalArgumentException();
            parent = new int[n]; size = new int[n]; components = n;
            for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        }
        public int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }
        public boolean union(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (size[a] < size[b]) { int t = a; a = b; b = t; }
            parent[b] = a; size[a] += size[b]; components--;
            return true;
        }
        public int size(int x) { return size[find(x)]; }
        public int components() { return components; }
    }

    /** Fenwick tree for point additions and half-open range sums. O(log n) per operation. */
    public static final class Fenwick {
        private final long[] tree;
        public Fenwick(int n) {
            if (n < 0) throw new IllegalArgumentException();
            tree = new long[n + 1];
        }
        public void add(int index, long delta) {
            if (index < 0 || index >= tree.length - 1) throw new IndexOutOfBoundsException();
            for (int i = index + 1; i < tree.length; i += i & -i) tree[i] += delta;
        }
        public long prefixSum(int endExclusive) {
            if (endExclusive < 0 || endExclusive >= tree.length) throw new IndexOutOfBoundsException
();
            long result = 0;
            for (int i = endExclusive; i > 0; i -= i & -i) result += tree[i];
            return result;
        }
        public long rangeSum(int left, int rightExclusive) {
            if (left > rightExclusive) throw new IllegalArgumentException();
            return prefixSum(rightExclusive) - prefixSum(left);
        }
    }

    /** Iterative segment tree for point assignment and half-open range sums. */
    public static final class SegmentTree {
        private final long[] tree;
        private final int n, base;
        public SegmentTree(long[] values) {
            n = values.length;
            int power = 1;
            while (power < n) power <<= 1;
            base = power;
            tree = new long[2 * base];
            for (int i = 0; i < n; i++) tree[base + i] = values[i];
            for (int i = base - 1; i > 0; i--) tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
        public void set(int index, long value) {
            if (index < 0 || index >= n) throw new IndexOutOfBoundsException();
            int p = base + index;
            tree[p] = value;
            for (p >>= 1; p > 0; p >>= 1) tree[p] = tree[2 * p] + tree[2 * p + 1];
        }
```

```
        public long sum(int left, int rightExclusive) {
            if (left < 0 || left > rightExclusive || rightExclusive > n) throw new
IndexOutOfBoundsException();
            long result = 0;
            for (int l = left + base, r = rightExclusive + base; l < r; l >>= 1, r >>= 1) {
                if ((l & 1) != 0) result += tree[l++];
                if ((r & 1) != 0) result += tree[--r];
            }
            return result;
        }
    }
}
```

3. Busqueda y ordenamiento

3.1. Busqueda binaria, ternaria y counting sort

Las busquedas requieren un arreglo ordenado.

```
package notebook;

/** Search and sorting routines. Input to searches must be sorted ascending. */
public final class SearchingSorting {
    private SearchingSorting() {}

    /** O(log n); returns any matching index, or -1. */
    public static int binarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }

    /** O(log n); returns any matching index, or -1. */
    public static int ternarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int m1 = lo + (hi - lo) / 3;
            int m2 = hi - (hi - lo) / 3;
            if (a[m1] == key) return m1;
            if (a[m2] == key) return m2;
            if (key < a[m1]) hi = m1 - 1;
            else if (key > a[m2]) lo = m2 + 1;
            else { lo = m1 + 1; hi = m2 - 1; }
        }
        return -1;
    }

    /** O(n + 65536) counting sort for Java UTF-16 char units. */
    public static void countingSort(char[] a) {
        int[] count = new int[Character.MAX_VALUE + 1];
        for (char c : a) count[c]++;
        int i = 0;
        for (int c = 0; c < count.length; c++) {
            while (count[c]-- > 0) a[i++] = (char) c;
        }
    }

    /** Lower bound: first index i with a[i] >= key. O(log n). */
    public static int lowerBound(int[] a, int key) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }
}
```

4. Programacion dinamica

4.1. Patrones de programacion dinamica

Incluye mochila 0/1, subsecuencias, cambio y particion.

```java
package notebook;

/** Small contest DP routines. Counts are long where practical and may overflow. */
public final class DynamicProgramming {
    private DynamicProgramming() {}

    /** Number of unordered combinations; positive coin values required. O(coins * target). */
    public static long coinChange(int[] coins, int target) {
        if (target < 0) return 0;
        long[] dp = new long[target + 1];
        dp[0] = 1;
        for (int coin : coins) {
            if (coin <= 0) throw new IllegalArgumentException("coin must be positive");
            for (int s = coin; s <= target; s++) dp[s] += dp[s - coin];
        }
        return dp[target];
    }

    /** Number of ordered ways to climb using steps of 1, 2, or 3. O(n). */
    public static long countWays(int n) {
        if (n < 0) return 0;
        long[] dp = new long[n + 1];
        dp[0] = 1;
        for (int i = 1; i <= n; i++)
            for (int step = 1; step <= 3; step++)
                if (i >= step) dp[i] += dp[i - step];
        return dp[n];
    }

    /** Levenshtein distance. O(a.length * b.length) time, O(b.length) space. */
    public static int editDistance(String a, String b) {
        int[] prev = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            int[] cur = new int[b.length() + 1];
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++)
                cur[j] = a.charAt(i - 1) == b.charAt(j - 1) ? prev[j - 1]
                    : 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
            prev = cur;
        }
        return prev[b.length()];
    }

    /** 0/1 knapsack with nonnegative weights and capacity. O(items * capacity). */
    public static long knapsack(int capacity, int[] weights, int[] values) {
        if (capacity < 0 || weights.length != values.length) throw new IllegalArgumentException();
        long[] dp = new long[capacity + 1];
        for (int i = 0; i < weights.length; i++) {
            if (weights[i] < 0) throw new IllegalArgumentException("negative weight");
            for (int w = capacity; w >= weights[i]; w--)
                dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
        return dp[capacity];
    }

    /** Longest common subsequence length. O(mn) time, O(n) space. */
    public static int lcs(String a, String b) {
        int[] dp = new int[b.length() + 1];
        for (int i = 1; i <= a.length(); i++) {
            int diag = 0;
            for (int j = 1; j <= b.length(); j++) {
                int old = dp[j];
                dp[j] = a.charAt(i - 1) == b.charAt(j - 1) ? diag + 1
                    : Math.max(dp[j], dp[j - 1]);
                diag = old;
            }
        }
        return dp[b.length()];
    }

    /** Strictly increasing subsequence length. O(n log n). */
    public static int lis(int[] a) {
```

```
        int[] tails = new int[a.length];
        int size = 0;
        for (int x : a) {
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = (lo + hi) >>> 1;
                if (tails[mid] < x) lo = mid + 1;
                else hi = mid;
            }
            tails[lo] = x;
            if (lo == size) size++;
        }
        return size;
    }

    /** Maximum product after at least one cut; n >= 2. O(n). */
    public static long maxProductCutting(int n) {
        if (n < 2) throw new IllegalArgumentException();
        if (n <= 3) return n - 1;
        long result = 1;
        while (n > 4) { result = Math.multiplyExact(result, 3); n -= 3; }
        return Math.multiplyExact(result, n);
    }

    /** Minimum difference between subset sums; nonnegative values. O(n * sum) time. */
    public static int minimumPartition(int[] a) {
        int sum = 0;
        for (int x : a) {
            if (x < 0) throw new IllegalArgumentException("negative value");
            sum = Math.addExact(sum, x);
        }
        boolean[] dp = new boolean[sum / 2 + 1];
        dp[0] = true;
        for (int x : a)
            for (int s = dp.length - 1; s >= x; s--) dp[s] |= dp[s - x];
        for (int s = dp.length - 1; s >= 0; s--) if (dp[s]) return sum - 2 * s;
        throw new AssertionError();
    }

    /** Subset sum for nonnegative values and target. O(n * target). */
    public static boolean subsetSum(int[] a, int target) {
        if (target < 0) return false;
        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for (int x : a) {
            if (x < 0) throw new IllegalArgumentException("negative value");
            for (int s = target; s >= x; s--) dp[s] |= dp[s - x];
        }
        return dp[target];
    }
}
```

5. Cadenas

5.1. KMP, Manacher, Boyer-Moore y Z

Opera sobre unidades UTF-16 de Java.

```
package notebook;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** String algorithms operate on Java UTF-16 code units. */
public final class StringsAlgorithms {
    private StringsAlgorithms() {}

    /** KMP match starts, including all n+1 positions for an empty pattern. O(n+m). */
    public static List<Integer> kmp(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        if (pattern.isEmpty()) {
            for (int i = 0; i <= text.length(); i++) matches.add(i);
            return matches;
        }
        int[] pi = prefixFunction(pattern);
        for (int i = 0, j = 0; i < text.length(); i++) {
```

```java
            while (j > 0 && text.charAt(i) != pattern.charAt(j)) j = pi[j - 1];
            if (text.charAt(i) == pattern.charAt(j)) j++;
            if (j == pattern.length()) { matches.add(i - j + 1); j = pi[j - 1]; }
        }
        return matches;
    }

    /** Prefix function. O(n). */
    public static int[] prefixFunction(String s) {
        int[] pi = new int[s.length()];
        for (int i = 1; i < s.length(); i++) {
            int j = pi[i - 1];
            while (j > 0 && s.charAt(i) != s.charAt(j)) j = pi[j - 1];
            if (s.charAt(i) == s.charAt(j)) j++;
            pi[i] = j;
        }
        return pi;
    }

    /** Longest palindromic substring by Manacher, O(n). */
    public static String manacher(String s) {
        if (s.isEmpty()) return "";
        int n = s.length(), bestStart = 0, bestLen = 1;
        int[] odd = new int[n], even = new int[n];
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 1 : Math.min(odd[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s.charAt(i - k) == s.charAt(i + k)) k++;
            odd[i] = k--;
            int len = 2 * odd[i] - 1;
            if (len > bestLen) { bestLen = len; bestStart = i - odd[i] + 1; }
            if (i + k > r) { l = i - k; r = i + k; }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 0 : Math.min(even[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s.charAt(i - k - 1) == s.charAt(i + k)) k++;
            even[i] = k--;
            int len = 2 * even[i];
            if (len > bestLen) { bestLen = len; bestStart = i - even[i]; }
            if (i + k > r) { l = i - k - 1; r = i + k; }
        }
        return s.substring(bestStart, bestStart + bestLen);
    }

    /** Boyer-Moore bad-character search. O(nm) worst case; UTF-16 aware. */
    public static List<Integer> boyerMoore(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        int n = text.length(), m = pattern.length();
        if (m == 0) {
            for (int i = 0; i <= n; i++) matches.add(i);
            return matches;
        }
        int[] last = new int[Character.MAX_VALUE + 1];
        Arrays.fill(last, -1);
        for (int i = 0; i < m; i++) last[pattern.charAt(i)] = i;
        for (int shift = 0; shift <= n - m;) {
            int j = m - 1;
            while (j >= 0 && pattern.charAt(j) == text.charAt(shift + j)) j--;
            if (j < 0) {
                matches.add(shift);
                shift += shift + m < n ? m - last[text.charAt(shift + m)] : 1;
            } else shift += Math.max(1, j - last[text.charAt(shift + j)]);
        }
        return matches;
    }

    /** Z-function: z[i] is longest common prefix of s and s.substring(i). O(n). */
    public static int[] zFunction(String s) {
        int[] z = new int[s.length()];
        for (int i = 1, l = 0, r = 0; i < s.length(); i++) {
            if (i <= r) z[i] = Math.min(r - i + 1, z[i - l]);
            while (i + z[i] < s.length() && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
            if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
        }
        return z;
    }
}
```

5.2. Trie y rolling hash

Hash doble; la igualdad de hashes no constituye una prueba exacta.

```java
package notebook;

import java.util.HashMap;
import java.util.Map;

/** Trie and double rolling hash over Java UTF-16 code units. */
public final class AdvancedStrings {
    private AdvancedStrings() {}

    public static final class Trie {
        private static final class Node {
            Map<Character, Node> children = new HashMap<>();
            int terminalCount;
        }
        private final Node root = new Node();

        /** Insert a word, including the empty word. O(length). */
        public void add(String word) {
            Node node = root;
            for (int i = 0; i < word.length(); i++)
                node = node.children.computeIfAbsent(word.charAt(i), key -> new Node());
            node.terminalCount++;
        }

        public int count(String word) {
            Node node = root;
            for (int i = 0; i < word.length(); i++) {
                node = node.children.get(word.charAt(i));
                if (node == null) return 0;
            }
            return node.terminalCount;
        }

        public boolean hasPrefix(String prefix) {
            Node node = root;
            for (int i = 0; i < prefix.length(); i++) {
                node = node.children.get(prefix.charAt(i));
                if (node == null) return false;
            }
            return true;
        }
    }

    /** O(n) build, O(1) half-open substring hash. Hash equality is probabilistic. */
    public static final class RollingHash {
        private static final long MOD1 = 1_000_000_007L, MOD2 = 1_000_000_009L;
        private static final long BASE = 911_382_323L;
        private final long[] h1, h2, p1, p2;
        public RollingHash(String s) {
            int n = s.length();
            h1 = new long[n + 1]; h2 = new long[n + 1];
            p1 = new long[n + 1]; p2 = new long[n + 1];
            p1[0] = p2[0] = 1;
            for (int i = 0; i < n; i++) {
                long c = s.charAt(i) + 1L;
                h1[i + 1] = (h1[i] * BASE + c) % MOD1;
                h2[i + 1] = (h2[i] * BASE + c) % MOD2;
                p1[i + 1] = p1[i] * BASE % MOD1;
                p2[i + 1] = p2[i] * BASE % MOD2;
            }
        }
        public long hash(int left, int rightExclusive) {
            if (left < 0 || left > rightExclusive || rightExclusive >= h1.length)
                throw new IndexOutOfBoundsException();
            int len = rightExclusive - left;
            long a = (h1[rightExclusive] - h1[left] * p1[len] % MOD1 + MOD1) % MOD1;
            long b = (h2[rightExclusive] - h2[left] * p2[len] % MOD2 + MOD2) % MOD2;
            return (a << 32) | b;
        }
    }
}
```

6. Matematicas

6.1. Teoria de numeros

Incluye primalidad determinista de 64 bits y CRT generalizado.

```java
package notebook;

import java.math.BigInteger;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/** Integer number theory. Modular operations require mod > 0. */
public final class NumberTheory {
    private NumberTheory() {}

    public static long gcd(long a, long b) {
        if (a == Long.MIN_VALUE || b == Long.MIN_VALUE)
            return BigInteger.valueOf(a).gcd(BigInteger.valueOf(b)).longValueExact();
        a = Math.abs(a); b = Math.abs(b);
        while (b != 0) { long t = a % b; a = b; b = t; }
        return a;
    }

    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        long product = Math.multiplyExact(a / gcd(a, b), b);
        if (product == Long.MIN_VALUE) throw new ArithmeticException("lcm does not fit in long");
        return Math.abs(product);
    }

    /** Returns [gcd(a,b), x, y] with ax+by=gcd; coefficients must fit in long. */
    public static long[] extendedGcd(long a, long b) {
        BigInteger aa = BigInteger.valueOf(a), bb = BigInteger.valueOf(b);
        BigInteger oldR = aa, r = bb, oldS = BigInteger.ONE, s = BigInteger.ZERO;
        BigInteger oldT = BigInteger.ZERO, t = BigInteger.ONE;
        while (r.signum() != 0) {
            BigInteger q = oldR.divide(r);
            BigInteger nr = oldR.subtract(q.multiply(r)); oldR = r; r = nr;
            BigInteger ns = oldS.subtract(q.multiply(s)); oldS = s; s = ns;
            BigInteger nt = oldT.subtract(q.multiply(t)); oldT = t; t = nt;
        }
        if (oldR.signum() < 0) { oldR = oldR.negate(); oldS = oldS.negate(); oldT = oldT.negate(); }
        return new long[] { oldR.longValueExact(), oldS.longValueExact(), oldT.longValueExact() };
    }

    /** Exact multiplication modulo mod without signed-long overflow. O(log b). */
    public static long modMul(long a, long b, long mod) {
        if (mod <= 0) throw new IllegalArgumentException("mod must be positive");
        long x = Math.floorMod(a, mod), y = Math.floorMod(b, mod), result = 0;
        while (y > 0) {
            if ((y & 1) != 0) result = addMod(result, x, mod);
            y >>>= 1;
            if (y > 0) x = addMod(x, x, mod);
        }
        return result;
    }

    private static long addMod(long a, long b, long mod) {
        return a >= mod - b ? a - (mod - b) : a + b;
    }

    /** a^exponent modulo mod. O(log exponent * log mod). */
    public static long modPow(long a, long exponent, long mod) {
        if (exponent < 0 || mod <= 0) throw new IllegalArgumentException();
        long result = 1 % mod, base = Math.floorMod(a, mod);
        while (exponent > 0) {
            if ((exponent & 1) != 0) result = modMul(result, base, mod);
            exponent >>>= 1;
            if (exponent > 0) base = modMul(base, base, mod);
        }
        return result;
    }

    /** Deterministic Miller-Rabin for signed 64-bit positive integers. */
    public static boolean isPrime(long n) {
        if (n < 2) return false;
```

```
        for (long p : new long[] {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
            if (n % p == 0) return n == p;
        }
        long d = n - 1;
        int s = Long.numberOfTrailingZeros(d);
        d >>= s;
        for (long witness : new long[] {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
            long a = witness % n;
            if (a == 0) continue;
            long x = modPow(a, d, n);
            if (x == 1 || x == n - 1) continue;
            boolean passed = false;
            for (int r = 1; r < s; r++) {
                x = modMul(x, x, n);
                if (x == n - 1) { passed = true; break; }
            }
            if (!passed) return false;
        }
        return true;
    }

    /** Primes <= n; O(n log log n) time, O(n) space. */
    public static List<Integer> sieve(int n) {
        if (n < 2) return new ArrayList<>();
        boolean[] composite = new boolean[n + 1];
        for (int p = 2; (long) p * p <= n; p++)
            if (!composite[p]) for (int x = p * p; x <= n; x += p) composite[x] = true;
        List<Integer> primes = new ArrayList<>();
        for (int i = 2; i <= n; i++) if (!composite[i]) primes.add(i);
        return primes;
    }

    /** Sorted positive divisors of n; n > 0. O(sqrt n). */
    public static List<Long> divisors(long n) {
        if (n <= 0) throw new IllegalArgumentException();
        List<Long> result = new ArrayList<>();
        for (long d = 1; d <= n / d; d++) {
            if (n % d == 0) {
                result.add(d);
                if (d != n / d) result.add(n / d);
            }
        }
        Collections.sort(result);
        return result;
    }

    /** Exact factorial; n >= 0. */
    public static BigInteger factorial(int n) {
        if (n < 0) throw new IllegalArgumentException();
        BigInteger result = BigInteger.ONE;
        for (int i = 2; i <= n; i++) result = result.multiply(BigInteger.valueOf(i));
        return result;
    }

    /** Generalized CRT. Returns [least nonnegative solution, lcm], or null if inconsistent. */
    public static BigInteger[] chineseRemainder(long[] moduli, long[] residues) {
        if (moduli.length != residues.length) throw new IllegalArgumentException();
        BigInteger x = BigInteger.ZERO, m = BigInteger.ONE;
        for (int i = 0; i < moduli.length; i++) {
            if (moduli[i] <= 0) throw new IllegalArgumentException("moduli must be positive");
            BigInteger next = BigInteger.valueOf(moduli[i]);
            BigInteger target = BigInteger.valueOf(residues[i]).mod(next);
            BigInteger g = m.gcd(next), delta = target.subtract(x);
            if (!delta.mod(g).equals(BigInteger.ZERO)) return null;
            BigInteger reduced = next.divide(g);
            BigInteger k = reduced.equals(BigInteger.ONE) ? BigInteger.ZERO
                : delta.divide(g).multiply(m.divide(g).modInverse(reduced)).mod(reduced);
            x = x.add(m.multiply(k));
            m = m.multiply(reduced);
            x = x.mod(m);
        }
        return new BigInteger[] { x, m };
    }

    /** Newton iteration; derivative must remain nonzero near the root. */
    public static double newtonRaphson(DoubleUnaryOperator f, DoubleUnaryOperator derivative,
                                       double start, double tolerance, int maxIterations) {
        if (!(tolerance > 0) || maxIterations <= 0) throw new IllegalArgumentException();
        double x = start;
```

```
        for (int i = 0; i < maxIterations; i++) {
            double fx = f.applyAsDouble(x), dx = derivative.applyAsDouble(x);
            if (Math.abs(fx) <= tolerance) return x;
            if (!Double.isFinite(dx) || dx == 0) throw new ArithmeticException("zero derivative");
            x -= fx / dx;
            if (!Double.isFinite(x)) throw new ArithmeticException("non-finite iterate");
        }
        throw new ArithmeticException("did not converge");
    }
}
```

6.2. Combinatoria

Resultados exactos con BigInteger.

```
package notebook;

import java.math.BigInteger;

public final class Combinatorics {
    private Combinatorics() {}

    /** Exact binomial coefficient. O(min(r,n-r)) big-integer operations. */
    public static BigInteger binomial(int n, int r) {
        if (r < 0 || r > n) return BigInteger.ZERO;
        r = Math.min(r, n - r);
        BigInteger ans = BigInteger.ONE;
        for (int i = 1; i <= r; i++)
            ans = ans.multiply(BigInteger.valueOf(n - i + 1)).divide(BigInteger.valueOf(i));
        return ans;
    }

    /** Catalan numbers C[0..n], exact. O(n) big-integer operations. */
    public static BigInteger[] catalan(int n) {
        if (n < 0) throw new IllegalArgumentException();
        BigInteger[] c = new BigInteger[n + 1];
        c[0] = BigInteger.ONE;
        for (int i = 1; i <= n; i++)
            c[i] = c[i - 1].multiply(BigInteger.valueOf(4L * i - 2)).divide(BigInteger.valueOf(i +
1));
        return c;
    }
}
```

7. Geometria

7.1. Geometria plana

Coordenadas double; usar predicados exactos cuando el problema lo exija.

```
package notebook;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

/** Planar geometry with double coordinates; use exact arithmetic for integer-critical predicates.
*/
public final class Geometry {
    private Geometry() {}

    public record Point(double x, double y) {}

    public static double distance(Point a, Point b) {
        return Math.hypot(a.x - b.x, a.y - b.y);
    }

    /** Shoelace area; input vertices are not repeated at the end. O(n). */
    public static double polygonArea(List<Point> p) {
        double twice = 0;
        for (int i = 0; i < p.size(); i++) {
            Point a = p.get(i), b = p.get((i + 1) % p.size());
            twice += a.x * b.y - a.y * b.x;
        }
```

```
        return Math.abs(twice) / 2;
    }

    /** Closed polygon perimeter; O(n). */
    public static double polygonPerimeter(List<Point> p) {
        double result = 0;
        for (int i = 0; i < p.size(); i++) result += distance(p.get(i), p.get((i + 1) % p.size()));
        return result;
    }

    private static double cross(Point a, Point b, Point c) {
        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    }

    /** Monotone-chain hull in counterclockwise order; removes collinear interior points. O(n log n). */
    public static List<Point> convexHull(Point[] points) {
        Point[] sorted = points.clone();
        Arrays.sort(sorted, Comparator.comparingDouble(Point::x).thenComparingDouble(Point::y));
        List<Point> unique = new ArrayList<>();
        for (Point p : sorted) if (unique.isEmpty() || !unique.get(unique.size() - 1).equals(p)) unique.add(p);
        if (unique.size() <= 1) return unique;
        List<Point> lower = new ArrayList<>(), upper = new ArrayList<>();
        for (Point p : unique) {
            while (lower.size() >= 2 && cross(lower.get(lower.size() - 2), lower.get(lower.size() - 1), p) <= 0)
                lower.remove(lower.size() - 1);
            lower.add(p);
        }
        for (int i = unique.size() - 1; i >= 0; i--) {
            Point p = unique.get(i);
            while (upper.size() >= 2 && cross(upper.get(upper.size() - 2), upper.get(upper.size() - 1), p) <= 0)
                upper.remove(upper.size() - 1);
            upper.add(p);
        }
        lower.remove(lower.size() - 1);
        upper.remove(upper.size() - 1);
        lower.addAll(upper);
        return lower;
    }

    /** Closest pair distance. O(n log^2 n); requires at least two points. */
    public static double closestPair(Point[] points) {
        if (points.length < 2) throw new IllegalArgumentException("at least two points required");
        Point[] sorted = points.clone();
        Arrays.sort(sorted, Comparator.comparingDouble(Point::x).thenComparingDouble(Point::y));
        return closest(sorted, 0, sorted.length);
    }

    private static double closest(Point[] p, int lo, int hi) {
        if (hi - lo <= 3) {
            double best = Double.POSITIVE_INFINITY;
            for (int i = lo; i < hi; i++) for (int j = i + 1; j < hi; j++) best = Math.min(best, distance(p[i], p[j]));
            return best;
        }
        int mid = (lo + hi) >>> 1;
        double splitX = p[mid].x;
        double best = Math.min(closest(p, lo, mid), closest(p, mid, hi));
        List<Point> strip = new ArrayList<>();
        for (int i = lo; i < hi; i++) if (Math.abs(p[i].x - splitX) < best) strip.add(p[i]);
        strip.sort(Comparator.comparingDouble(Point::y));
        for (int i = 0; i < strip.size(); i++)
            for (int j = i + 1; j < strip.size() && strip.get(j).y - strip.get(i).y < best; j++)
                best = Math.min(best, distance(strip.get(i), strip.get(j)));
        return best;
    }

    /** Diameter via convex hull and rotating calipers. O(n log n). */
    public static double farthestPair(Point[] points) {
        List<Point> hull = convexHull(points);
        int n = hull.size();
        if (n < 2) return 0;
        if (n == 2) return distance(hull.get(0), hull.get(1));
        int j = 1;
        double best = 0;
        for (int i = 0; i < n; i++) {
```

```
            int next = (i + 1) % n;
            while (Math.abs(cross(hull.get(i), hull.get(next), hull.get((j + 1) % n)))
                    > Math.abs(cross(hull.get(i), hull.get(next), hull.get(j)))) j = (j + 1) % n;
            best = Math.max(best, Math.max(distance(hull.get(i), hull.get(j)), distance(hull.get(
next), hull.get(j))));
        }
        return best;
    }
}
```

8. Grafos

8.1. Recorridos y caminos

Grafos dirigidos por defecto; agregar ambas aristas para no dirigidos.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/** Zero-indexed graph algorithms. */
public final class Graphs {
    private Graphs() {}

    /** Directed adjacency list. For undirected graphs add both directions. */
    public static List<List<Integer>> newGraph(int n) {
        if (n < 0) throw new IllegalArgumentException();
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        return graph;
    }

    /** Unweighted shortest-path distances; unreachable vertices have -1. O(V+E). */
    public static int[] bfs(List<List<Integer>> graph, int start) {
        int[] dist = new int[graph.size()];
        Arrays.fill(dist, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        dist[start] = 0; queue.add(start);
        while (!queue.isEmpty()) {
            int v = queue.remove();
            for (int to : graph.get(v)) if (dist[to] == -1) {
                dist[to] = dist[v] + 1; queue.add(to);
            }
        }
        return dist;
    }

    /** Iterative DFS visitation order. O(V+E). */
    public static List<Integer> dfs(List<List<Integer>> graph, int start) {
        boolean[] seen = new boolean[graph.size()];
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        stack.push(start);
        while (!stack.isEmpty()) {
            int v = stack.pop();
            if (seen[v]) continue;
            seen[v] = true; order.add(v);
            List<Integer> neighbors = graph.get(v);
            for (int i = neighbors.size() - 1; i >= 0; i--) if (!seen[neighbors.get(i)]) stack.push(
neighbors.get(i));
        }
        return order;
    }

    /** Kahn topological order; throws if the directed graph contains a cycle. */
    public static List<Integer> topologicalSort(List<List<Integer>> graph) {
        int[] indegree = new int[graph.size()];
        for (List<Integer> edges : graph) for (int to : edges) indegree[to]++;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < indegree.length; i++) if (indegree[i] == 0) queue.add(i);
        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
```

```
            int v = queue.remove(); result.add(v);
            for (int to : graph.get(v)) if (--indegree[to] == 0) queue.add(to);
        }
        if (result.size() != graph.size()) throw new IllegalArgumentException("graph has a cycle");
        return result;
    }

    public record WeightedEdge(int to, long weight) {}

    /** Dijkstra; requires nonnegative weights. Unreachable vertices have Long.MAX_VALUE. */
    public static long[] dijkstra(List<List<WeightedEdge>> graph, int start) {
        long[] dist = new long[graph.size()];
        Arrays.fill(dist, Long.MAX_VALUE);
        PriorityQueue<long[]> queue = new PriorityQueue<>(Comparator.comparingLong(a -> a[0]));
        dist[start] = 0; queue.add(new long[] {0, start});
        while (!queue.isEmpty()) {
            long[] current = queue.remove();
            long d = current[0]; int v = (int) current[1];
            if (d != dist[v]) continue;
            for (WeightedEdge edge : graph.get(v)) {
                if (edge.weight < 0) throw new IllegalArgumentException("negative edge");
                if (d > Long.MAX_VALUE - edge.weight) continue;
                long candidate = d + edge.weight;
                if (candidate < dist[edge.to]) {
                    dist[edge.to] = candidate;
                    queue.add(new long[] {candidate, edge.to});
                }
            }
        }
        return dist;
    }

    public record Edge(int from, int to, long weight) {}

    /** Kruskal MST; throws if disconnected. O(E log E). */
    public static long kruskal(int n, List<Edge> edges) {
        List<Edge> sorted = new ArrayList<>(edges);
        sorted.sort(Comparator.comparingLong(Edge::weight));
        DataStructures.DSU dsu = new DataStructures.DSU(n);
        long total = 0;
        for (Edge edge : sorted) if (dsu.union(edge.from, edge.to)) total = Math.addExact(total,
edge.weight);
        if (n > 0 && dsu.components() != 1) throw new IllegalArgumentException("disconnected graph")
;
        return total;
    }
}
```

8.2. SCC, Bellman-Ford, LCA y flujo maximo

Algoritmos de grafos avanzados; capacidades y distancias usan long.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** Additional graph routines. Vertices are zero-indexed. */
public final class AdvancedGraphs {
    private AdvancedGraphs() {}

    /** Kosaraju SCC labels for a directed graph. O(V+E), iterative DFS. */
    public static int[] stronglyConnectedComponents(List<List<Integer>> graph) {
        int n = graph.size();
        List<List<Integer>> reverse = Graphs.newGraph(n);
        for (int v = 0; v < n; v++) for (int to : graph.get(v)) reverse.get(to).add(v);
        boolean[] seen = new boolean[n];
        List<Integer> order = new ArrayList<>();
        for (int start = 0; start < n; start++) {
            if (seen[start]) continue;
            ArrayDeque<int[]> stack = new ArrayDeque<>();
            seen[start] = true; stack.push(new int[] {start, 0});
            while (!stack.isEmpty()) {
                int[] frame = stack.peek();
                List<Integer> edges = graph.get(frame[0]);
                if (frame[1] == edges.size()) { order.add(frame[0]); stack.pop(); continue; }
```

```
                int to = edges.get(frame[1]++);
                if (!seen[to]) { seen[to] = true; stack.push(new int[] {to, 0}); }
            }
        }
        int[] component = new int[n];
        Arrays.fill(component, -1);
        int count = 0;
        for (int i = order.size() - 1; i >= 0; i--) {
            int start = order.get(i);
            if (component[start] != -1) continue;
            ArrayDeque<Integer> stack = new ArrayDeque<>();
            stack.push(start); component[start] = count;
            while (!stack.isEmpty()) {
                for (int to : reverse.get(stack.pop())) if (component[to] == -1) {
                    component[to] = count; stack.push(to);
                }
            }
            count++;
        }
        return component;
    }

    /** Bellman-Ford; throws if a reachable negative cycle exists. O(VE). */
    public static long[] bellmanFord(int n, List<Graphs.Edge> edges, int source) {
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        dist[source] = 0;
        for (int pass = 0; pass < n; pass++) {
            boolean changed = false;
            for (Graphs.Edge edge : edges) {
                if (dist[edge.from()] == Long.MAX_VALUE) continue;
                long candidate = Math.addExact(dist[edge.from()], edge.weight());
                if (candidate < dist[edge.to()]) {
                    if (pass == n - 1) throw new IllegalArgumentException("reachable negative cycle"
);
                    dist[edge.to()] = candidate;
                    changed = true;
                }
            }
            if (!changed) break;
        }
        return dist;
    }

    /** Binary-lifting LCA for a connected, undirected tree. O(n log n) build, O(log n) query. */
    public static final class LCA {
        private final int[][] up;
        private final int[] depth;

        public LCA(List<List<Integer>> tree, int root) {
            int n = tree.size(), levels = 1;
            while ((1L << levels) <= Math.max(1, n)) levels++;
            up = new int[levels][n]; depth = new int[n];
            Arrays.fill(depth, -1);
            ArrayDeque<Integer> queue = new ArrayDeque<>();
            depth[root] = 0; up[0][root] = root; queue.add(root);
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (int to : tree.get(v)) if (depth[to] == -1) {
                    depth[to] = depth[v] + 1; up[0][to] = v; queue.add(to);
                }
            }
            for (int d : depth) if (d < 0) throw new IllegalArgumentException("disconnected tree");
            for (int k = 1; k < levels; k++)
                for (int v = 0; v < n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
        }

        public int query(int a, int b) {
            if (depth[a] < depth[b]) { int t = a; a = b; b = t; }
            int delta = depth[a] - depth[b];
            for (int k = 0; delta > 0; k++, delta >>= 1) if ((delta & 1) != 0) a = up[k][a];
            if (a == b) return a;
            for (int k = up.length - 1; k >= 0; k--) if (up[k][a] != up[k][b]) {
                a = up[k][a]; b = up[k][b];
            }
            return up[0][a];
        }
    }
```

```java
    /** Dinic maximum flow with nonnegative long capacities. */
    public static final class Dinic {
        private static final class Arc {
            int to, reverse;
            long capacity;
            Arc(int to, int reverse, long capacity) {
                this.to = to; this.reverse = reverse; this.capacity = capacity;
            }
        }
        private final List<List<Arc>> graph = new ArrayList<>();
        private final int[] level, next;
        public Dinic(int n) {
            if (n < 0) throw new IllegalArgumentException();
            for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
            level = new int[n]; next = new int[n];
        }
        public void addEdge(int from, int to, long capacity) {
            if (capacity < 0) throw new IllegalArgumentException();
            int forwardReverse = graph.get(to).size() + (from == to ? 1 : 0);
            Arc forward = new Arc(to, forwardReverse, capacity);
            Arc backward = new Arc(from, graph.get(from).size(), 0);
            graph.get(from).add(forward); graph.get(to).add(backward);
        }
        private boolean levels(int source, int sink) {
            Arrays.fill(level, -1);
            ArrayDeque<Integer> queue = new ArrayDeque<>();
            queue.add(source); level[source] = 0;
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (Arc arc : graph.get(v)) if (arc.capacity > 0 && level[arc.to] == -1) {
                    level[arc.to] = level[v] + 1; queue.add(arc.to);
                }
            }
            return level[sink] >= 0;
        }
        private long send(int v, int sink, long flow) {
            if (v == sink) return flow;
            List<Arc> arcs = graph.get(v);
            for (; next[v] < arcs.size(); next[v]++) {
                Arc arc = arcs.get(next[v]);
                if (arc.capacity == 0 || level[arc.to] != level[v] + 1) continue;
                long pushed = send(arc.to, sink, Math.min(flow, arc.capacity));
                if (pushed == 0) continue;
                arc.capacity -= pushed;
                Arc back = graph.get(arc.to).get(arc.reverse);
                back.capacity = Math.addExact(back.capacity, pushed);
                return pushed;
            }
            return 0;
        }
        public long maxFlow(int source, int sink) {
            if (source == sink) return 0;
            long total = 0;
            while (levels(source, sink)) {
                Arrays.fill(next, 0);
                long pushed;
                while ((pushed = send(source, sink, Long.MAX_VALUE)) > 0) total = Math.addExact(
total, pushed);
            }
            return total;
        }
    }
}
```

8.3. Puentes y puntos de articulacion

Admite aristas paralelas; DFS recursivo.

```java
package notebook;

import java.util.ArrayList;
import java.util.List;

/** Bridges and articulation points in an undirected multigraph. Recursive DFS uses O(V) stack. */
public final class UndirectedCuts {
    private UndirectedCuts() {}

    public record Edge(int u, int v) {}
```

```java
    public record Result(List<Integer> bridges, List<Integer> articulationPoints) {}

    /** Bridges are returned as indices into edges. O(V+E). */
    public static Result find(int n, List<Edge> edges) {
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int i = 0; i < edges.size(); i++) {
            Edge edge = edges.get(i);
            graph.get(edge.u).add(new int[] {edge.v, i});
            graph.get(edge.v).add(new int[] {edge.u, i});
        }
        int[] tin = new int[n], low = new int[n];
        boolean[] cut = new boolean[n];
        List<Integer> bridges = new ArrayList<>();
        int[] timer = {1};
        for (int v = 0; v < n; v++) if (tin[v] == 0)
            dfs(v, -1, graph, tin, low, cut, bridges, timer);
        List<Integer> cuts = new ArrayList<>();
        for (int i = 0; i < n; i++) if (cut[i]) cuts.add(i);
        bridges.sort(null);
        return new Result(bridges, cuts);
    }

    private static void dfs(int v, int parentEdge, List<List<int[]>> graph, int[] tin, int[] low,
                            boolean[] cut, List<Integer> bridges, int[] timer) {
        tin[v] = low[v] = timer[0]++;
        int children = 0;
        for (int[] edge : graph.get(v)) {
            int to = edge[0], id = edge[1];
            if (id == parentEdge) continue;
            if (tin[to] != 0) low[v] = Math.min(low[v], tin[to]);
            else {
                dfs(to, id, graph, tin, low, cut, bridges, timer);
                low[v] = Math.min(low[v], low[to]);
                if (low[to] > tin[v]) bridges.add(id);
                if (parentEdge != -1 && low[to] >= tin[v]) cut[v] = true;
                children++;
            }
        }
        if (parentEdge == -1 && children > 1) cut[v] = true;
    }
}
```

9. Teoria de juegos

9.1. Nim, Grundy y minimax

Nim normal y estados Grundy aciclicos.

```java
package notebook;

import java.util.HashSet;
import java.util.Set;

public final class GameTheory {
    private GameTheory() {}

    /** Normal-play Nim: first player wins iff pile xor is nonzero. */
    public static boolean firstWinsNim(int[] piles) {
        int xor = 0;
        for (int pile : piles) {
            if (pile < 0) throw new IllegalArgumentException();
            xor ^= pile;
        }
        return xor != 0;
    }

    /** Minimum excluded nonnegative integer. */
    public static int mex(Set<Integer> values) {
        int result = 0;
        while (values.contains(result)) result++;
        return result;
    }

    /** Grundy values for a DAG whose edges only lead to lower-numbered states. */
    public static int[] grundy(int[][] moves) {
        int[] result = new int[moves.length];
```

```
        for (int i = 0; i < moves.length; i++) {
            Set<Integer> reachable = new HashSet<>();
            for (int to : moves[i]) {
                if (to < 0 || to >= i) throw new IllegalArgumentException("moves must lead to lower states");
                reachable.add(result[to]);
            }
            result[i] = mex(reachable);
        }
        return result;
    }

    /** Minimax of a complete binary game tree; leaves length must be a power of two. */
    public static int minimax(int[] leaves) {
        if (leaves.length == 0 || (leaves.length & (leaves.length - 1)) != 0)
            throw new IllegalArgumentException("leaf count must be a power of two");
        int[] level = leaves.clone();
        boolean maximize = (Integer.numberOfTrailingZeros(leaves.length) & 1) == 1;
        while (level.length > 1) {
            int[] next = new int[level.length / 2];
            for (int i = 0; i < next.length; i++)
                next[i] = maximize ? Math.max(level[2 * i], level[2 * i + 1])
                    : Math.min(level[2 * i], level[2 * i + 1]);
            level = next;
            maximize = !maximize;
        }
        return level[0];
    }
}
```